Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

When using a list becomes painful

Today
Welcome to your next lesson: When using a list becomes painful.

I'm preparing your personalized lesson based on what you've learned so far, what’s next for you, and the best way to help you practice.

Step by step, you’re moving forward.

Imagine your phone’s contacts were stored in a plain numbered list like this:

Alice – 555-1111
Bob – 555-2222
Carol – 555-3333
Dave – 555-4444

You want Carol’s number. With a simple list like this, you only know positions (1, 2, 3, 4…), not who is at each position. So to find Carol, you’d have to scan: check 1, then 2, then 3… until you hit “Carol”.

This is called position-based retrieval:

You get things by their position (index) in the list: 0, 1, 2, 3…

Make sense so far — that with a plain list, you only really “know” positions, not names?

Yes, that makes sense

What do you think? Take your time with the question above 👆

You have a list of student names in Python:

students = ["Alice", "Bob", "Carol", "Dave"]
If you only know that Carol is in the list, but you don’t know her index, how do you conceptually find her?

A.

Look at each item from the start until you see "Carol"

B.

Jump straight to position 2 because lists know names

C.

Ask Python: "Give me the item with name 'Carol'" directly, without scanning

D.

You can’t find "Carol" in a list at all

Nice, exactly right 🎯

In a plain Python list like students = ["Alice", "Bob", "Carol", "Dave"], the list only understands positions (0, 1, 2, 3…). It has no built-in idea of “the one named Carol”. So to find Carol when you don’t know her index, you conceptually:

Start at the beginning and check each item one by one until you see "Carol".

That “walk through the list, checking each item” is the key pattern I want you to feel.

Next step: let’s flip it.

Imagine you’re writing a program that needs to look up dozens of students by name during a run — over and over.

Would you want to keep scanning the list from the start each time, or does that already feel a bit annoying? 😅

That sounds annoying

Exactly — that “ugh, again?” feeling is the signal we care about here 😅

Let’s make it concrete.

Say you have:

Python

```
students = ["Alice", "Bob", "Carol", "Dave"]
grades   = [85,      92,    78,      90]
```

So:

index 0 → Alice, 85
index 1 → Bob, 92
index 2 → Carol, 78
index 3 → Dave, 90

To get Carol’s grade, the steps are:

Scan students from the start until you find "Carol".
Remember the index where you found her (here, 2).
Use that index on grades[2] to get 78.

So every time you want a grade by name, you have to do this little dance: search → remember index → use index.

Does that 3-step process make sense so far: search in one list, grab index, then use that index in another list?

Yes, that's clear

What do you think is the main pain here? Take your time with the question above 👇

Imagine your program needs to look up dozens of student grades by name while it runs.

Right now you have two lists:

```
students = ["Alice", "Bob", "Carol", "Dave", ...]
grades   = [85,      92,    78,     90,    ...]
```

Every time you get a name like "Carol", you:

Scan students until you find that name
Remember the index
Use that index on grades
What's the main pain with using lists this way?

A.

You must keep scanning the list from the start over and over to find the right index

B.

You can only store numbers in lists, not names and grades

C.

Python will randomly change the order of the list while your program runs

D.

You can't ever get a grade from a list — lists are only for printing data

Exactly 🎯

The big pain is that you must keep scanning from the start to find the index every time you have a name. That means:

Lots of repeated work (search, search, search…)
More code to write and read
Easier to make mistakes (mix up indexes, wrong pairings, etc.)

Your instinct here is spot on — this is where lists start to feel "not quite the right tool".

Now I want to switch how we think about this.

Instead of:

"Find Carol's position, then get the grade at that position"

Imagine you could just say:

"Give me the grade for the student whose name is "Carol"."

This is called key-based retrieval:

You use a meaningful label (the key, like "Carol") to jump straight to the value you want (her grade), instead of walking through positions.

Make sense so far:

Position-based → use index numbers
Key-based → use a meaningful label like a name?
Yes, that makes sense

Great 👍

Let’s do a quick real-world picture.

Think of a set of labeled drawers:

The label on the drawer = the key (like a student’s name)
The thing inside = the value (like their grade)

With key-based retrieval, you don’t open every drawer. You just:

Look at the label “Carol” → open that one drawer → you’re done 🎯

With a plain list, it’s like having numbered drawers only (0, 1, 2, 3…) and no names on the front. To find Carol, you’d have to open drawer 0, then 1, then 2… until you find her.

Ready to see a tiny Python example where this list-based searching starts to feel annoying?

Yes, let's see it

Nice 🙌 let’s keep it simple and visual.

Here’s a small Python program that uses lists to look up a price by product name:

We’ll imagine this structure:

Python

```
products = ["apple", "banana", "orange", "grape"]
prices   = [0.99,     0.59,     0.79,    2.49]
```

Goal: when someone types "orange", we want to show its price.

To do that with lists, we have to:

Start at index 0 in products
Walk forward until we find the matching name
Use that same index in prices

I’d like you to code this in the code editor:

Use the products and prices lists above
Ask the user for a product name with input()
Loop through products to find the name
When you find it, print the matching price from prices

Don’t worry about handling “not found” yet.

👉 Go ahead and write this in the editor, run it, then Submit code so I can see it.

Python

```
products = ["apple", "banana", "orange", "grape"]
prices   = [0.99,     0.59,     0.79,    2.49]



product_name = input("Please enter a product: ")
for i in (0, len(products)-1):
    if products[i] == product_name:
        print(f"The price is {prices[i]}")
```

Nice work getting that running, Ron 🎯

When you typed orange, your program correctly printed 2.49 — so the basic idea is right.

There's one small but important detail in your loop:

Python

```
for i in (0, len(products)-1):
```

This doesn't loop through all indexes, it only checks i = 0 and i = len(products)-1 (so just the first and last item). By luck, "grape" still works, but "banana" or "orange" only worked here because you tested once and the name happened to be at one of those positions when you tried.

To actually scan the whole list, you want:

Python

```
for i in range(len(products)):
```

That means: 0, 1, 2, 3 — check every drawer.

This is exactly the "pain pattern" we're studying:

you have to write a loop that walks index by index, checking each name, just to get one price.

Does it make sense why range(len(products)) is what lets you really search the whole list?

it got grape the last one

Right, and that's a great catch 👍

Your loop:

Python

```
for i in (0, len(products)-1):
```

only checks:

i = 0 → "apple"
i = len(products)-1 → "grape"

So:

If you type apple → it works ☑
If you type grape → it works ☑
But if you type banana or orange → the loop never even looks at those positions, so it

won’t find them ✖

To really scan the whole list, you need something like:

Python

```
for i in range(len(products)):
```

That gives you 0, 1, 2, 3 → checks all products.

This is a good moment to notice the pain:

You had to manage an index
You had to remember to loop over the full range
It’s easy to accidentally skip items (like here)

That’s exactly why list-based lookups start to feel fragile when done a lot.

Want to try one quick change?

Update your loop to use range(len(products)), run it, test with apple, orange, and grape, then hit Submit code again so I can check it.

Python

```
products = ["apple", "banana", "orange", "grape"]
prices   = [0.99,     0.59,     0.79,    2.49]



product_name = input("Please enter a product: ")
for i in range(len(products)):
    if products[i] == product_name:
        print(f"The price is {prices[i]}")
```

Nice fix, Ron 🎯

Now your loop really walks the entire list:

Python

```
for i in range(len(products)):
    if products[i] == product_name:
        print(f"The price is {prices[i]}")
```

Step by step, this does:

range(len(products)) → makes the indexes 0, 1, 2, 3
For each i, it checks: “Is products[i] the name the user typed?”
When it finds a match, it uses the same index i to grab the price from prices[i]

So if the user types "orange":

At i = 2, products[2] is "orange" → match ☑
Then it prints prices[2], which is 0.79

This works — but you can probably feel how much work it is just to map name → price.

I want to check your intuition now.

Imagine you’re building a bigger app where, all the time, you need to answer questions like:

“What’s the price of 'orange'?”
“What’s the price of 'banana'?”

“What’s the price of 'grape'?”

Over and over, many times.

Would you prefer to:

keep doing this search-through-lists-by-index pattern, or
have something where you can directly ask for a price by product name, like using the drawer label?

Which one feels like the better long-term tool to you?

Directly ask by name

What do you think fits best here? Take your time with the question above 👆

You’re designing a Python program for a small grocery store.

You often need to answer questions like: "What’s the price of 'milk'?" or "What’s the price of 'eggs'?" — many times while the program runs.

Which structure is a better fit for storing the data you’ll look up by product name all the time?

A.

A simple list where you always search through items from the start to find the name you want

B.

A structure that works like labeled drawers: each product name is a label (key) that points straight to its price (value)

C.

Many separate variables like milk_price, eggs_price, bread_price written all over the code

D.

A list that you print once at the start of the program, but never use again

Exactly the right instinct 🎯

When your questions are naturally “What’s the price of this name?”, a structure that works like labeled drawers is the best fit:

Each product name is a label (the key) that points straight to its price (the value).

That avoids:

scanning a list from the start every time
juggling indexes between two separate lists
subtle bugs when indexes get out of sync

This “label → value” idea is the heart of key-based retrieval, and in Python that’s what a dictionary is for (we’ll get into the syntax later).

Let’s lock in the main idea with one last quick choice:

You now know:

Lists shine when you care about order and positions (0, 1, 2, 3…)
Lookup tables (like dictionaries) shine when you care about labels/keys (names, IDs, product codes)

In your own words, which situation do you personally feel is a clear signal that a list is the wrong tool and you should reach for a key-based structure instead?

Just describe one example, like “when I keep…”.

you need to search more then 1 time

Beautifully said 👌

Needing to search more than once is a big red flag:

If you keep writing code that scans the same list again and again just to find things by a meaningful label (like name, product, ID), that’s a strong signal you really want a key-based structure instead.

That usually shows up as:

repeated loops that look almost the same
lots of if item == target: checks
maybe even multiple parallel lists tied together by index
You’ve basically hit both mastery goals here:

You can spot when list-based lookups get awkward 👍
You can justify why a key-based “labeled drawers” structure is better in those cases 👍

You’ve mastered what this lesson wanted. 🎓

Ready to wrap up this lesson and move on to the next one?

Yes, I'm ready

Nice work on this one, Ron 🎯

Here’s what you nailed in this lesson:

You saw why lists get painful when you need to keep searching them by some label (name, product, etc.) instead of by position.
You felt the cost of that pattern in code: loops, index juggling, and easy mistakes.
You understood that when your question is “What’s the value for this name/ID/key?”, a key-based structure (like our labeled drawers / a Python dictionary) is a much better fit than a plain list.
Core rule you discovered:

If you keep searching a list by some label more than once, consider switching to a lookup table (dictionary) where that label is the key.

You’re set up really well for the next step, where we’ll start actually using that key-based structure in Python.

If anything from this lesson still feels fuzzy (lists vs keys, the pain points, etc.), tell me and we’ll clear it up before you dive into the next one. 🍸

```
products = ["apple", "banana", "orange", "grape"]
prices   = [0.99,     0.59,     0.79,    2.49]
```

```
511
512
513
514   product_name = input("Please enter a product: ")
515   for i in range(len(products)):
516       if products[i] == product_name:
517           print(f"The price is {prices[i]}")
518
```